SAILAB against other flood models: a controlled case study

Same terrain, rainfall, friction and scoring for every model, scored against the UNOSAT satellite flood map of the 2022 Sindh floods.

Generated 31 October 2026. SAILAB v6.1, LISFLOOD-FP 8.2.

1. What this shows, in plain terms

> **SAILAB's accuracy is on par with the other open 2D flood models, not better.** Across three test areas SAILAB's mean CSI is 0.504, against 0.503 (ANUGA), 0.497 (TELEMAC-2D) and 0.500 (SFINCS). The differences are about 0.01 to 0.03 and the three areas disagree on the ordering, so there is no winner on accuracy. SAILAB and ANUGA flooded almost the same cells (94% overlap), and TELEMAC-2D matched SAILAB at 90%. Independent codes agreeing this closely is good evidence that SAILAB's solver is correct.

> **No model, including SAILAB, scored clearly better than the trivial "flood everything" guess** (mean CSI 0.500). Raising the wet-depth cut-off makes every model score worse (section 5). The most likely reason is the forcing, not the models: the 2022 Sindh flood was mostly river overflow and upstream water, and this test applies only local rainfall. This has not been tested directly.

Where SAILAB does have a measured advantage:

- **Speed.** On the same cases SAILAB's GPU solver took 58 s on average, against 141 s (TELEMAC-2D), 528 s (SFINCS) and 477 s (ANUGA). The comparison is GPU against CPU, ANUGA and TELEMAC used a coarser 90 m mesh, and TELEMAC ran on one core, so it shows what a user gets, not a like-for-like solver speed test.
- **Time from question to result.** From a drawn polygon to a finished result with buildings, roads, risk and loss took SAILAB 81 to 187 seconds with nothing prepared by hand (section 7). The other open models needed hand-written converters of 59 to 114 lines each.
- **GPU.** SAILAB's solver runs on the GPU. No GPU support for HEC-RAS 2D was found in the public release notes I could read (section 8).
- **Integration and cost.** SAILAB runs the whole workflow (rainfall, buildings, roads, loss, scenarios, monitoring) on one machine with no licence fee. None of the other models do that on their own.
- **Portability.** SAILAB's solver runs on this ARM GPU server; the commercial and Windows tools in section 9 could not be run here at all.

Where it is not ahead: accuracy (parity), depth validation (none for any model here), and the volume of published validation behind HEC-RAS, TUFLOW, MIKE and others.

2. Study design

- **Event and observed map:** the 2022 Pakistan floods, Sindh. Observed flood extent from UNOSAT (VIIRS satellite, 29 November 2022). The map is from three months after the August peak, so it shows water that persisted, not the rain-driven peak.
- **Areas:** three 13 km boxes placed along the edge of the observed flood so each is about half flooded (49%, 50%, 51%). An earlier test box that was 98% flooded was dropped because it cannot show skill.
- **Same inputs for every model:** SRTM elevation (29 m), 253.5 mm of rain over 48 hours (the wettest 48-hour ERA5 window at the site), Manning n = 0.05, no infiltration, closed boundaries, nothing calibrated.
- **Scoring:** cells counted wet above 5 cm peak depth. CSI = hits / (hits + misses + false alarms); POD = share of observed flooded cells found; FAR = share of predicted wet cells that were dry in the observation.
- **Baselines:** "Flood everything", and "lowest terrain", which floods the lowest cells up to the true flooded fraction (it is told the answer's size).
- **Models run:** SAILAB (LISFLOOD-FP 8.2, ACC solver, GPU); ANUGA 4.1 (full shallow-water, finite volume, 90 m mesh); SFINCS (Deltares, local-inertial, built from source); TELEMAC-2D (full shallow-water, finite element, built from source, 90 m mesh, constant rain at the same total).

3. Results

Model	CSI A	CSI B	CSI C	Mean CSI	POD	FAR	Wet area / observed	Mean run time	Hardware
Flood everything	0.486	0.505	0.509	**0.500**	1.00	0.50	2.00x	-	-
Lowest terrain (oracle fraction)	0.392	0.507	0.347	**0.415**	0.58	0.42	1.00x	-	-
SAILAB	0.504	0.528	0.481	**0.504**	0.89	0.46	1.65x	58 s	GPU (NVIDIA GB10)
ANUGA	0.495	0.535	0.478	**0.503**	0.86	0.45	1.57x	477 s	CPU, 90 m mesh
SFINCS	0.486	0.505	0.509	**0.500**	1.00	0.50	2.00x	528 s	CPU, 20 threads, 29 m
TELEMAC-2D	0.490	0.531	0.469	**0.497**	0.84	0.45	1.53x	141 s	CPU, 1 core, 90 m mesh

Areas: A Manchar edge, B north Dadu, C Shikarpur. Italic rows are the trivial baselines. Wet area / observed above 1 means over-prediction.

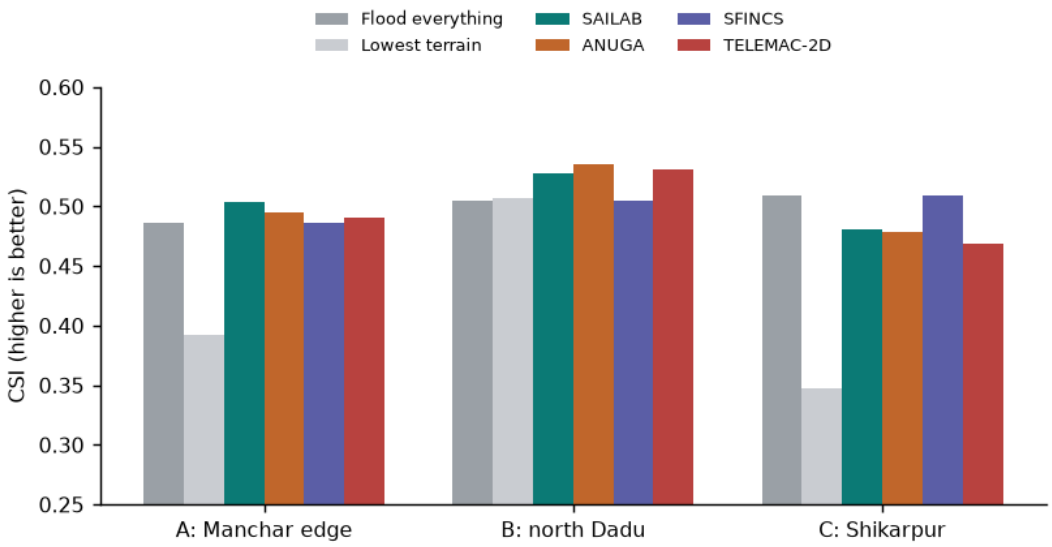


Figure 1. CSI by area. The grey bars are the baselines.

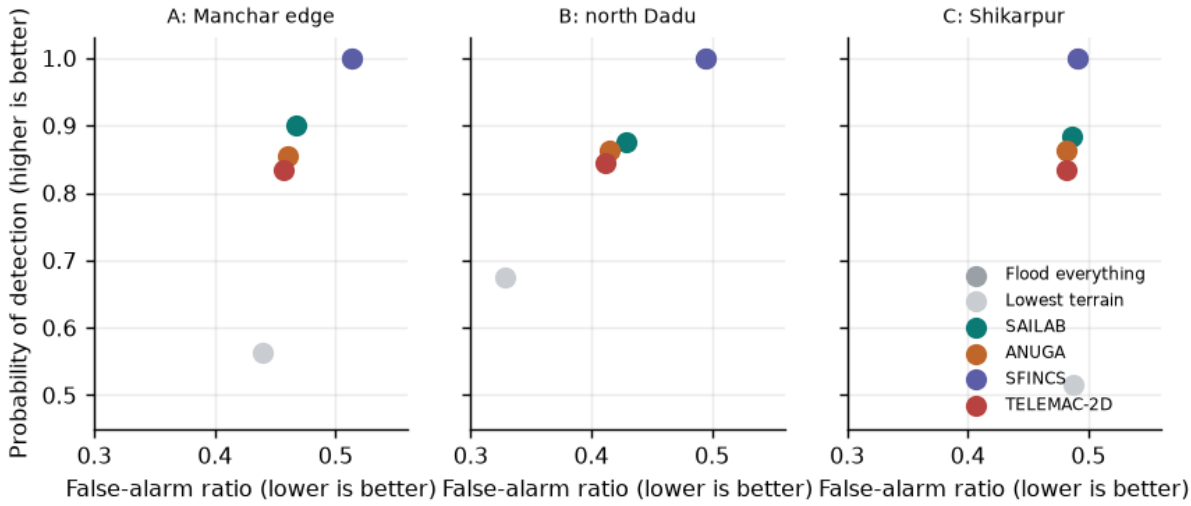
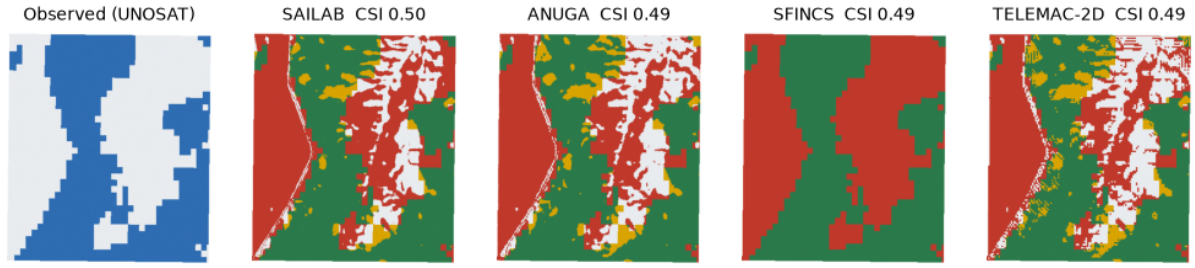


Figure 2. Detection against false alarms. All four models find 83 to 100% of the flooded cells; SFINCS reaches 100% only because it reports more than 5 cm of water in every cell.

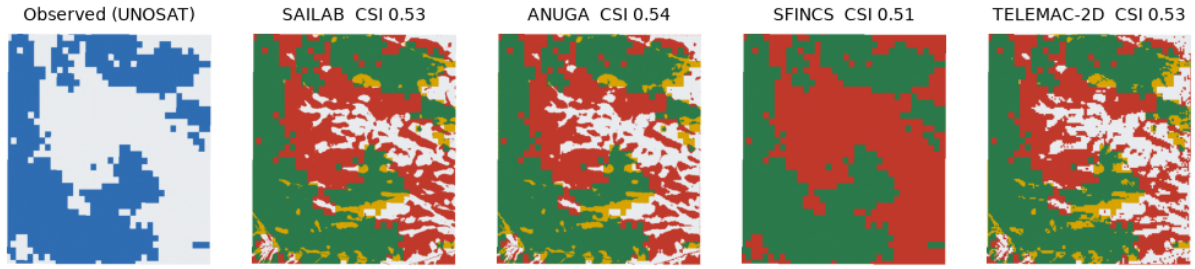
4. Maps

Key: correctly wet missed false alarm correctly dry

Area A: Manchar edge



Area B: north Dadu



Area C: Shikarpur

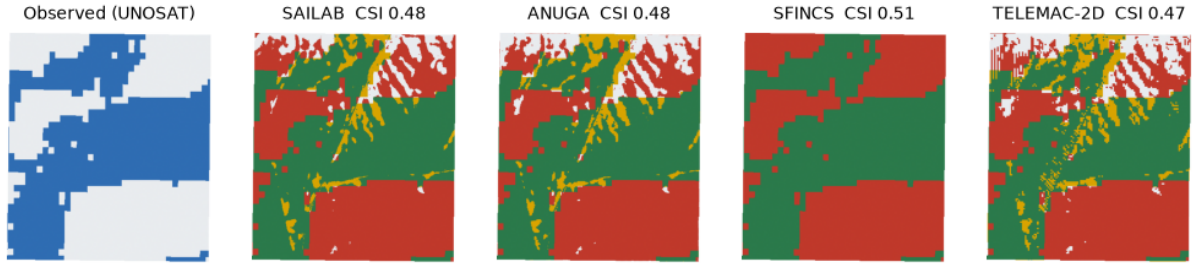


Figure 3. Observed flooding (left, blue) and each model's result against it. All four models flood the low western strip of area A, which the satellite map shows as dry.

Peak depth, area A

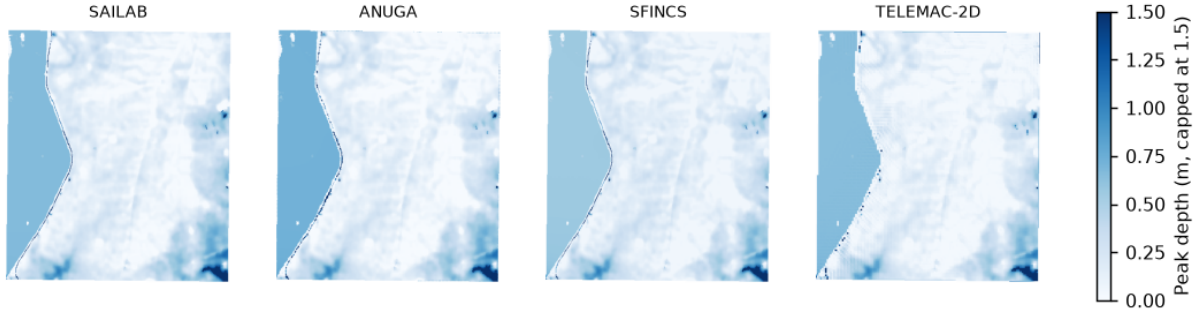


Figure 4. Peak depth from each model (same scale). The patterns are alike; maximum depths are 4.1 to 4.5 m.

5. How robust is the result

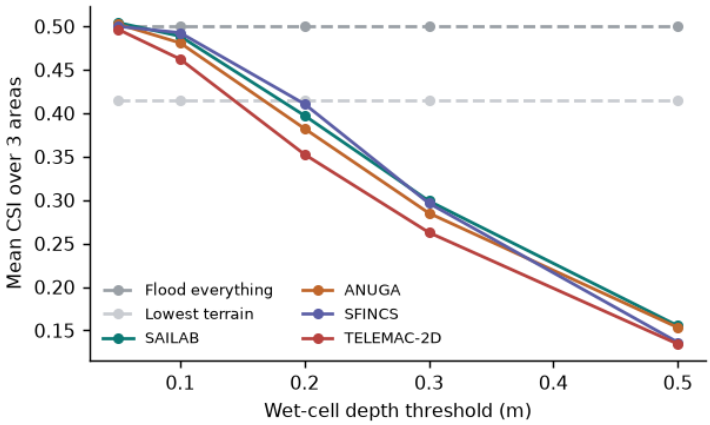


Figure 5. Mean CSI as the wet-depth cut-off rises. Every model falls below "flood everything" at every cut-off from 10 cm up, and below both baselines from 20 cm up (SAILAB beats "flood everything" only at 5 cm, by 0.004), so the models' depth patterns do not line up with the observed extent. A model that is right for the right reasons would hold up better as the cut-off rises.

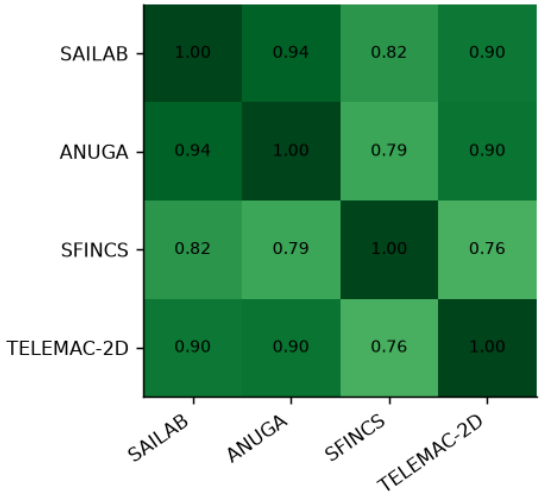


Figure 6. How much the models' flooded cells overlap each other (1 = identical). SAILAB, ANUGA and TELEMAC-2D agree at 0.90 to 0.94; SFINCS differs most because it keeps a thin layer of water everywhere.

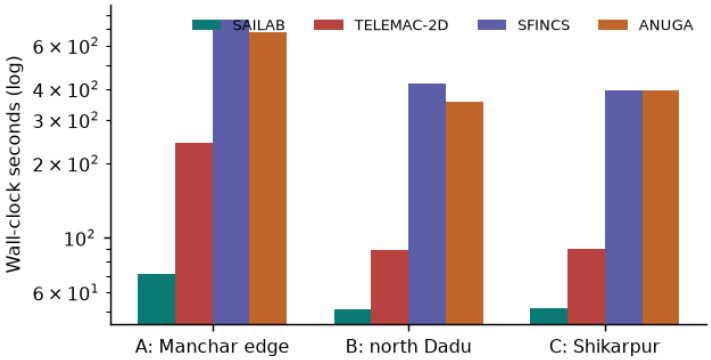


Figure 7. Run times (log scale). Hardware differs between models, see section 1.

6. What the evidence supports

- SAILAB's solver gives the same flood as independent codes on the same inputs: this is the strongest finding, and it supports the correctness of the numerics.
- It cannot yet claim better real-world accuracy than the other models, and this test alone cannot show whether any of them has real skill on this flood.
- SAILAB ran about 2 to 9 times faster than the CPU models on these cases (TELEMAC-2D 2.4x, ANUGA 8x, SFINCS 9x on average), with the hardware caveat above.

> **Limits of this study.** One event; three small areas; observed map three months after the peak; rain-only forcing that cannot represent river overflow; no depth data to compare against; meshes of 29 m and 90 m differ between models; each model's own numerical settings were left at defaults; the area-A west strip may reflect SRTM elevation error. None of these favour or disadvantage SAILAB specifically.

7. Time from question to result

Solver run time is only part of the work. The tools differ more in what a user must prepare before the solver can start.

What SAILAB took, measured

Drawing a 13 km area and pressing run, with everything fetched and processed automatically (elevation, land-cover friction, buildings burned in as obstacles, roads, facilities, flood solver, building and road risk, estimated loss, 3D scene), on the live server:

Area	Total, polygon to finished result	Of which flood solver	Buildings used
A_manchar_edge	**187 s**	81 s	6,023
B_north_dadu	**81 s**	52 s	10,394
C_shikarpur	**87 s**	53 s	5,959

Measured 31 Oct 2026, fresh runs with no cache, 48 h storm. No files were prepared by hand.

What the other models needed from me, measured

For the three open models I ran, I had to write the converters myself, because none of them reads a DEM and a rainfall total directly. Script lengths, excluding the shared scoring code:

Model	Lines of code I wrote	What it had to do
SAILAB	**0**	Polygon in, result out.
ANUGA	59	Build a triangular mesh, interpolate elevation onto it, set friction, boundaries and a rain function, collect maximum depth.
SFINCS	69	Write elevation and mask grids, a rainfall table and the input file; read the result back.
TELEMAC-2D	114	Write a mesh file in its binary format and a boundary file by hand, write the steering file, read the result back.

Each also needed its own installation, and I hit and fixed a wrong-package install (SFINCS), a parallel-build race, a missing output keyword and a wrong output setting along the way. That is not a fair measure of an expert's time, but it shows these tools start from a terrain file and little else.

What the commercial tools typically require (general knowledge, not measured here)

Input	HEC-RAS 2D, TUFLOW, MIKE and similar	SAILAB
Terrain	User sources, clips and loads a DEM	Built in (Pakistan SRTM); cropped to the drawn polygon
Mesh or grid	User builds and checks it	Automatic
Roughness		Automatic from ESA WorldCover

	User maps land cover to Manning values	
Buildings, roads, facilities	Separate datasets, collected and joined by the user	Built in (Google Open Buildings, road and facility layers)
Rainfall and inflow	User prepares time series	Entered, uploaded, or taken from forecasts and design-storm data
Boundary conditions	User defines them	Defaults, or river detection and inflow tools
Damage and loss	Separate tool or spreadsheet	Built in

I did not time these workflows and have no measured figure for how long they take; the point is the number of separate steps, not hours.

8. Hardware and GPU support

Model	GPU support	Basis
SAILAB (LISFLOOD-FP)	**Yes, CUDA.** Runs on the NVIDIA GB10 in this server.	Measured: 51 to 71 s per area on the GPU
LISFLOOD-FP (published)	Yes; the uniform solver is 2 to 28 times faster than its CPU version, the non-uniform solver up to 320 times	LISFLOOD-FP 8.1 paper, Geoscientific Model Development 2023
HEC-RAS 2D	No GPU support found in the public release notes I could access (not confirmed against the official HEC notes)	Search, 31 Oct 2026
TUFLOW	Yes (TUFLOW GPU / HPC)	Listed in the UK Environment Agency 2013 benchmark
TRITON, SERGHEI, RIM2D, HiPIMS	Yes (multi-GPU)	Published papers; not run here
SRH-2D	Yes, new in 2026; average speedup about 7.7x in one deck	Aquaveo webinar
ANUGA, TELEMAC-2D, SFINCS (as run here)	CPU in this study (TELEMAC and ANUGA builds are CPU)	Measured: 89 to 767 s per area

SAILAB's speed in section 3 therefore compares a GPU solver with CPU solvers. That is a real advantage of the product on the hardware it was built for, not proof that the underlying numerical scheme is faster. Models that were not run (HEC-RAS and others) were not timed, so no speed claim is made against them.

9. Models not run, and why

Model	Why it was not run	What that means for SAILAB
HEC-RAS 2D	Only Windows and x86 Linux builds exist; this server is ARM. A 2D case also needs a mesh/	Free, but cannot run here. SAILAB runs on this machine with no licence or Windows.

	geometry file normally built in its Windows GUI.	
TUFLOW (Classic/HPC/FV)	Commercial licence required, Windows/x86.	No licence cost for SAILAB's solver (GPL).
MIKE 21 / MIKE+	Commercial licence (DHI), Windows.	Same.
InfoWorks ICM	Commercial licence (Autodesk), Windows.	Same.
Flood Modeller	Commercial licence, Windows.	Same.
FLO-2D	Windows desktop program; Pro tier is paid.	Same.
Delft3D-FM / D-Flow FM	Official binaries are x86; no ARM package found.	SAILAB is built from source for this ARM/GPU machine.
TRITON, SERGHEI, RIM2D, HiPIMS	Need source builds; GPU support on this ARM GPU is untested. Not attempted for time.	Not a SAILAB advantage: untested.
Fathom, Google Flood Hub, GloFAS	Data products or services, not runnable models.	Published scores only (see survey).

Not running a model is not evidence that it is worse or better than SAILAB. What it does show is a practical difference: SAILAB's solver is open and runs on this ARM GPU server without a licence or Windows, while most of the established tools need a licence, Windows or x86 hardware. For these models the survey (SAILAB_Flood_Model_Survey.md) lists published validation results instead; HEC-RAS, for example, has been benchmarked against LISFLOOD-FP (the engine inside SAILAB) on the Secchia River flood, where both mis-estimated extent and depth in built-up areas.

10. What published studies found when they compared HEC-RAS, LISFLOOD-FP and SWAT

SAILAB's flood engine is LISFLOOD-FP, so published HEC-RAS versus LISFLOOD-FP studies are the most relevant outside evidence. They use different events, metrics and settings, so their numbers cannot be compared with this report's CSI. "Checked" means I read the paper's own text; the rest rest on the abstract or a search summary and should be checked before quoting.

Study	Models and site	What it found	Checked
Shustikova et al. 2019, Hydrological Sciences Journal	HEC-RAS 5.0.3 (2D) vs LISFLOOD-FP, Secchia River, Italy, 25 to 100 m grids	25 m grids were best for both; 50 and 100 m performed nearly the same. Both mis-estimated extent and depth in built-up areas.	Abstract
Fernandez and Mourato, River Lis, Portugal	HEC-RAS vs LISFLOOD-FP, 4 km reach, calibrated on the October 2006 flood, driven by HEC-HMS	For the 100-year flood the two models' inundated areas overlapped by 65%. HEC-RAS flood-wave travel time was about half LISFLOOD-FP's, and LISFLOOD-FP took much longer to run.	Yes

Oikonomou et al., Thessaly, Greece	HEC-RAS (1D), LISFLOOD-FP and FLO-2D (quasi-2D), Penios basin	Run times were about 3 h (LISFLOOD-FP), 2 h (FLO-2D) and 10 min (HEC-RAS). Outflow depth in HEC-RAS varied from 6.05 to 19.47 m as floodplain roughness went from 0.03 to 0.3, against 5.65 to 8.31 m for LISFLOOD-FP.	Yes
Gorgan Plain, Iran (via author page)	1D HEC-RAS vs 2D LISFLOOD-FP against aerial imagery	F-statistic 0.69 (LISFLOOD-FP) vs 0.67 (HEC-RAS); LISFLOOD-FP slightly better in validation.	No
Horritt and Bates 2002, River Severn, UK	HEC-RAS, TELEMAC-2D, LISFLOOD-FP	HEC-RAS and TELEMAC-2D could be calibrated to give good predictions; LISFLOOD-FP needed independent inundated-area data to perform acceptably.	No
EGU 2009 abstract, River Alzette, Luxembourg	HEC-RAS vs LISFLOOD-FP	LISFLOOD-FP represented complex floodplain flow well but ran slower; HEC-RAS described in-channel flow in more detail and ran much faster.	Abstract
HEC-RAS on the Indus: Naeem et al. 2021 (Tori levee breach, 2010) and other studies	HEC-RAS only, no LISFLOOD-FP	2D HEC-RAS matched observed stages within about 0.13 to -0.54 m and 84% of maximum inundation area for the 2010 breach. A 1D model of the Chashma to Kot Mithon reach reported NSE 0.92 to 0.95.	No

SWAT

I found no study comparing SWAT with LISFLOOD or LISFLOOD-FP. They answer different questions: SWAT is a catchment water-balance model that predicts river flow, not flood extent, and "LISFLOOD" (the JRC hydrological model) is a different program from LISFLOOD-FP. Related results found: SWAT against HEC-HMS on the Kalu River, Sri Lanka (SWAT NSE 0.87 calibration and 0.66 validation; HEC-HMS 0.91 and 0.56); SWAT on the Upper Indus Basin (calibration NSE 0.81 to 0.86, doing worse than a neural-network model); and JRC LISFLOOD on the Wei River, China (daily NSE 0.69, monthly 0.79). All of these come from search summaries and are unchecked. SAILAB's closest counterpart to a catchment model is its snowmelt module, which scored KGE 0.86 against GloFAS on the Hunza basin with default settings; GloFAS is itself a model, not gauge data.

> **What the literature adds.** Published comparisons tend to find HEC-RAS much faster and more detailed in the channel, and LISFLOOD-FP better at spreading water across the floodplain, with HEC-RAS showing larger sensitivity to floodplain roughness in one study. None found one clearly more accurate than the other overall. That fits this report's own result: SAILAB's LISFLOOD-FP engine performs about like other 2D models, and nothing published or measured here shows it ahead on accuracy.

11. Full survey table

All the models considered, with whether each fact came from a source retrieved during this work (S) or from general knowledge not re-verified (K).

Model	Type / physics	Licence, cost	GPU	Scale seen in literature	Validation evidence found	Basis
SAILAB (LISFLOOD-FP 8.2 inside)	2D local-inertial (ACC) or DG2 shallow-water; plus HAND instant preview, coastal bathtub/ HVAR, GLOF, snowmelt, loss	Own app; solver GPL	Yes (CUDA, GB10)	Pakistan-wide, no size cap	Case study in this report: three half-flooded areas, 2022 Sindh, mean CSI 0.50, on par with ANUGA, SFINCS and TELEMAC-2D and the trivial baseline. (An earlier CSI of 0.66 came from a 98%-flooded box and is withdrawn.)	own
LISFLOOD-FP 8.x	2D local-inertial / non-uniform grid	GPL v3, non-commercial per a source	Yes; ACC 2-28x faster than CPU, non-uniform up to 320x	Regional	Many papers; 8.3 released 13 Oct 2026	S
HEC-RAS 2D	Full SWE or diffusion wave	Free (US Army Corps)	Not verified	Reach to basin	Benchmarked against the UK EA suite (Brunner 2018); compared with LISFLOOD-FP on the Secchia River (both misestimated extent/depth in built-up zones)	S
		Commercial, by module	Yes (TUFLOW GPU)	Urban to catchment	In the UK EA 2013 benchmark	S

TUFLOW (Classic / HPC / FV)	Full SWE; HPC GPU solver				of 2D packages	
MIKE 21 / MIKE+ / MIKE FLOOD	Full SWE, flexible mesh	Commercial	Not verified	Coastal to urban	In the EA 2013 benchmark ensemble	S
InfoWorks ICM	1D/2D integrated, urban drainage	Commercial (Autodesk)	Not verified	Urban	In the EA ensemble	S
Flood Modeller	1D/2D	Commercial	Not verified	River	Not found in this pass	S/K
FLO-2D	Diffusive / dynamic wave	$995/yr Pro, free Basic tier (2020 source, may be outdated)	No (K)	Floodplain	US FEMA-accepted (K)	S/K
SOBEK	1D/2D	Commercial/ free versions (K)	No (K)	River	In the EA benchmark	S
ANUGA	Full SWE, finite volume	Open source	Not verified	Regional	In the EA ensemble	S
TELEMAC-2D	Full SWE, finite element	Open source	No (K)	River to estuary	Compared with HEC-RAS (Loire; Indonesia flood, 2026 preprint)	S
Delft3D FM / D-Flow FM	Full SWE, flexible mesh, coastal	Open source (K)	In development (K)	Coast to basin	Used as reference for SFINCS	S/K
SFINCS	Reduced-physics (inertial), subgrid; compound flooding	Open source (K)	Not verified	Coast to global	About 100x faster than Delft3D-FLOW on a Hurricane Irma case (thesis); tested on 499 global river events (HESS 2026)	S
RIM2D	Local-inertia, GPU	Open (K)	Yes	Urban to catchment	Compared with SERGHEI on the 2021	S

					Ahr flood: similar accuracy at coarse resolution, differences at fine	
SERGHEI	Full SWE, GPU/HPC	Open source	Yes (multi-GPU)	Catchment	Compared with RIM2D (above)	S
TRITON	Full SWE, first-order upwind, multi-GPU	Open source (ORNL)	Yes	6,800 km2, 68 M cells, 10-day storm in under 30 min	Published test cases	S
HiPIMS	Full SWE, GPU	Open source (K)	Yes (K)	Catchment	No result retrieved	K
Fathom (US and global 30 m)	Terrain + LISFLOOD-FP-based hazard maps	Commercial data product	n/a (precomputed)	Continental / global	Hit rate 86% vs FEMA maps (US, 2017); CSI about 0.75 and water-level error about 0.6 m (global 30 m, 2024)	S
Google Flood Hub	LSTM river forecast + inundation model	Free service, not a runnable model	n/a	150+ countries	5,680 training gauges; compared against GloFAS v4	S
GloFAS	Hydrological discharge forecast, not inundation	Free data	n/a	Global	Used as the benchmark in this app's melt checks	S/K
SWMM (EPA)	1D urban drainage with 2D coupling options	Free	No	Urban	Widely used (K)	K

12. Recommended next steps to make the accuracy case

1. Score an event where the flood can be reproduced from its drivers: known river inflow (for example the 2010 Indus flood with gauge hydrographs), or a rain-driven urban flood with a mapped extent.
2. Add observed depths (high-water marks, gauges) so depth can be scored, not just extent.
3. Run SAILAB with river inflow from GloFAS on the 2022 areas, to test the river-overflow explanation.
4. Run HEC-RAS or TUFLOW on the same three areas on a licensed x86 machine, and score with the same code (inputs are available).
5. Try 8.3 of LISFLOOD-FP on a multi-million-cell basin; on our test cases it matched 8.2 and showed no speed gain.

13. Reproducibility

All code and inputs are in `~/flood/cmp`: `case_common.py` (shared DEM, observed map and scoring), `run_baselines_sailab.py`, `run_anuga.py`, `run_sfincs.py`, `run_telemac.py`, `score_all.py`, `report_build.py`. Raw numbers: `results_all.json`, `results_extra.json`. Models: SAILAB/LISFLOOD-FP 8.2; ANUGA 4.1.0 (pip); SFINCS v2.5.0-beta (Deltares, built from source 31 Oct 2026); TELEMAC v9.2-dev (built from source, serial). The first SFINCS attempt used a conda package with the same name that turned out to be a plasma-physics code; it was removed and no result came from it. The earlier SAILAB test reporting CSI 0.66 used a box that was 98% flooded and should not be quoted as accuracy.